



Aplicaciones e Industria de IoT

Semana 2 — Conectividad, Plataformas y Arquitecturas IoT

KOICA

IGU HANDONG GLOBAL
UNIVERSITY



UNA

ÍNDICE

1. **Protocolos de Comunicación IoT: MQTT, HTTP, CoAP**
2. **Tecnologías de Red para IoT**
3. **Plataformas IoT: Nube, Código Abierto y Comparativa**
4. **Diseño de Arquitecturas IoT: Metodología y Patrones**
5. **Laboratorio PBL: Arquitectura MQTT Completa**
6. **Resumen, Bibliografía y Próximos Pasos**



Repaso de la Semana 1 y Objetivos de la Semana 2

Lo que aprendimos en la Semana 1

- Las 4 capas de una arquitectura IoT (Dispositivo → Red → Plataforma → Aplicación).
- Diferencias críticas entre IoT de consumo e IIoT industrial.
- Industria 4.0: 9 pilares, CPS y modelo de madurez.
- Verticales: manufactura, ciudades, agro, logística, salud.
- Edge/Fog/Cloud, seguridad (OWASP IoT Top 10) y tendencias.
- LAB: Nodo IoT ambiental con ESP32 + DHT22 en Wokwi.

Objetivos de la Semana 2

- Comprender protocolos IoT clave: MQTT, HTTP/REST, CoAP.
- Comparar tecnologías de red: Wi-Fi, BLE, LoRaWAN, NB-IoT, 5G.
- Evaluar plataformas IoT en la nube y de código abierto.
- Diseñar una arquitectura IoT de alto nivel para el proyecto integrador.
- Implementar comunicación MQTT real desde ESP32 en Wokwi.

Conexión con la Semana 1: Hoy profundizamos en las Capas 2 y 3 de la arquitectura IoT

4

Aplicación

(Semanas 4-8)

3

Plataforma

HOY: Plataformas IoT

2

Red

HOY: Protocolos y conectividad

1

Dispositivo

(Semana 1: ESP32 + sensores)

1. PROTOCOLOS DE COMUNICACIÓN IoT

MQTT · HTTP/REST · CoAP

¿Por Qué Importan los Protocolos en IoT?

Un protocolo de comunicación es el conjunto de reglas que define cómo dos dispositivos intercambian datos: formato del mensaje, secuencia de envío, confirmaciones y manejo de errores. Elegir el protocolo correcto determina la eficiencia, confiabilidad y escalabilidad de toda la solución IoT.



Los 3 Protocolos Clave en IoT



MQTT

Message Queuing Telemetry Transport

Protocolo ligero de mensajería basado en el modelo publicar/suscribir. Diseñado para redes inestables y dispositivos con recursos limitados. Es el estándar de facto en IoT.

Puerto: 1883 / 8883 (TLS)

Modelo: Pub/Sub



HTTP/REST

Hypertext Transfer Protocol

Protocolo de solicitud/respuesta que todos conocemos de la web. Robusto y universal pero más pesado que MQTT. Ideal para APIs, dashboards y comunicación con servicios web.

Puerto: 80 / 443 (TLS)

Modelo: Request/Response



CoAP

Constrained Application Protocol

Versión ligera de HTTP diseñada para dispositivos muy restringidos. Usa UDP en vez de TCP, con mensajes compactos. Ideal para sensores de bajo consumo en redes LPWAN.

Puerto: 5683 / 5684 (DTLS)

Modelo: Request/Response

MQTT – El Protocolo Estrella del IoT

Historia y Contexto

- Creado en 1999 por Andy Stanford-Clark (IBM) y Arlen Nipper para monitorear oleoductos en el desierto con conexión satelital limitada.
- Diseñado para: ancho de banda mínimo, alta latencia, conexiones inestables y dispositivos con poca memoria.
- Estándar OASIS desde 2014. Versión actual: MQTT 5.0.
- Usado por: AWS IoT, Azure IoT Hub, Facebook Messenger, Tesla, y miles de soluciones industriales.

Características Técnicas

- Cabecera mínima: solo 2 bytes (vs. ~700 bytes en HTTP).
- Transporte: TCP/IP (confiable, orientado a conexión).
- Modelo: Publicar/Suscribir con un Broker central.
- Bidireccional: dispositivos envían y reciben mensajes.
- Soporta TLS/SSL para comunicación cifrada (puerto 8883).

Modelo Publicar/Suscribir (Pub/Sub) – Cómo funciona MQTT

PUBLICADOR (Publisher)

Sensor ESP32 que envía datos de temperatura al topic: planta/sala-servers/temp

PUBLISH →

BROKER MQTT (Servidor Central)

Recibe mensajes, los filtra por topic y los reenvía a todos los clientes suscritos.
Ejemplos: Mosquitto, EMQX, HiveMQ.

→ DELIVER

SUSCRIPTOR (Subscriber)

Dashboard o aplicación que recibe los datos en tiempo real. Puede ser ThingsBoard, Node-RED, app móvil, etc.

MQTT – Topics, Jerarquías y Comodines

Un topic MQTT es la dirección a la que se publican y suscriben los mensajes. Se organiza como una ruta jerárquica separada por barras (/). NO se crean previamente: se generan al publicar el primer mensaje.

Estructura Jerárquica de Topics

empresa / ubicación / área / dispositivo / variable

Ejemplos:

claro/asuncion/datacenter/rack-A3/temperatura
agro/campo-norte/sector-2/sensor-suelo/humedad
fabrica/villeta/linea-1/motor-01/vibracion
hospital/ips/uci/cama-12/spo2

Comodines (Wildcards)

+ (un nivel): **sustituye exactamente un nivel.**

`fabrica/villeta/+/motor-01/temp`

→ Recibe de cualquier línea, solo motor-01

(multilevel): **sustituye todos los niveles restantes.**

`fabrica/villeta/#`

→ Recibe TODOS los mensajes de la planta Villeta

Buenas Prácticas para Diseñar Topics MQTT

- Usar minúsculas y guiones (no espacios ni caracteres especiales). Ejemplo: sensor-temp-01 en vez de Sensor Temp 01.
- Estructura: general → específico (empresa/sitio/área/dispositivo/variable). Facilita filtrado con comodines.
- Separar telemetría de comandos: .../telemetry vs .../command. Nunca mezclar datos de lectura con instrucciones de control.
- Evitar topics demasiado genéricos ("datos") o demasiado largos (más de 5 niveles suele ser excesivo).

MQTT – Calidad de Servicio (QoS)



Niveles de Calidad de Servicio (QoS)

QoS 0

A lo sumo una vez

El mensaje se envía una sola vez sin confirmación. Puede perderse. Más rápido y ligero.

Uso: Telemetría frecuente donde perder una lectura no es crítico (ej: temp cada 2 seg).

QoS 1

Al menos una vez

El broker confirma recepción (PUBACK). Si no confirma, se reenvía. Puede haber duplicados.

Uso: Alertas y datos importantes. La aplicación debe tolerar mensajes duplicados.

QoS 2

Exactamente una vez

Protocolo de 4 pasos garantiza entrega única. Más lento y consume más ancho de banda.

Uso: Transacciones críticas: comandos de parada de emergencia, facturación, registros legales.



Mensajes Retenidos (Retained)

- Cuando un publicador envía un mensaje con el flag retained = true, el broker guarda la última copia.
- Cualquier nuevo suscriptor que se conecte al topic recibe inmediatamente el último valor retenido.
- Útil para: estado actual del dispositivo, última lectura de sensor, configuración.
- Sin retained, un suscriptor nuevo no ve nada hasta que llega el próximo mensaje.



Última Voluntad (Last Will & Testament)

- Al conectarse, el dispositivo registra un mensaje LWT en el broker: "si me desconecto inesperadamente, publica este mensaje".
- Si el dispositivo se desconecta sin enviar DISCONNECT, el broker publica automáticamente el LWT.
- Ejemplo: topic planta/motor-01/status con payload {"status":"OFFLINE"} permite detectar dispositivos caídos.
- Esencial para monitoreo de salud de la red de sensores.

HTTP/REST y CoAP – Alternativas a MQTT en IoT

HTTP/REST en IoT

- Modelo solicitud/respuesta: el cliente pide, el servidor contesta.
- Verbos: GET (leer), POST (crear), PUT (actualizar), DELETE (borrar).
- Payload típico: JSON. Cabeceras de ~700 bytes por solicitud.
- Ventaja: universal, bien documentado, compatible con cualquier lenguaje.
- Desventaja: pesado para dispositivos con poca memoria/batería. No es bidireccional nativo (requiere WebSocket).
- Uso en IoT: APIs de plataformas cloud, dashboards web, configuración de dispositivos, webhooks.

CoAP – HTTP Ligero para IoT Restringido

- Diseñado por el IETF (RFC 7252) para dispositivos muy limitados.
- Usa UDP (no TCP): más ligero pero sin garantía de entrega nativa.
- Cabecera de solo 4 bytes (vs. 2 de MQTT, vs. ~700 de HTTP).
- Soporta observación (Observe): similar a suscripción para actualizaciones.
- Seguridad con DTLS (versión de TLS para UDP).
- Uso: sensores en redes 6LoWPAN, dispositivos con memoria <10KB, redes mesh muy restringidas.

Tabla Comparativa – ¿Cuándo Usar Cada Protocolo?

Criterio	MQTT	HTTP/REST	CoAP
Transporte	TCP	TCP	UDP
Cabecera	2 bytes	~700 bytes	4 bytes
Modelo	Pub/Sub	Req/Resp	Req/Resp + Observe
Bidireccional	Sí (nativo)	No (WebSocket)	Sí (Observe)
Consumo energía	Bajo	Alto	Muy bajo
Caso IoT ideal	Telemetría, alertas	APIs, dashboards	Sensores 6LoWPAN

Ejercicio – Conectar ESP32 a un Broker MQTT en Wokwi

EJERCICIO EN CLASE



Objetivo: Publicar datos del DHT22 vía MQTT desde el ESP32 simulado en Wokwi



Instrucciones Paso a Paso

1. Abrir wokwi.com → ESP32 → nuevo proyecto.
2. Montar circuito del LAB 1 (DHT22 + 3 LEDs).
3. En la pestaña Libraries, agregar:
 - DHT sensor library
 - PubSubClient (librería MQTT para Arduino)
4. Copiar el código de la siguiente diapositiva.
5. Ejecutar simulación y abrir Serial Monitor.
6. Verificar que aparezca: "Conectado al broker MQTT" y los mensajes publicados.
7. Abrir MQTTX (cliente) y suscribirse al topic para ver los datos en tiempo real.



¿Qué Van a Aprender?

- Cómo un ESP32 se conecta a un broker MQTT por Wi-Fi.
- La diferencia entre `Serial.print` (local) y `mqtt.publish` (remoto).
- Cómo definir un topic MQTT jerárquico.
- Cómo verificar los datos con un cliente MQTT externo.



Datos del Broker Público

Servidor: broker.hivemq.com
Puerto: 1883 (sin cifrado)
No requiere usuario ni contraseña
Topic sugerido: `iot-fpuna/TU-NOMBRE/sensor`

Cliente para verificar: MQTTX (descarga gratuita)
o MQTT Explorer

Ejercicio – Código: ESP32 + MQTT en Arduino IDE

Código Completo – Copiar en sketch.ino de Wokwi

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT22
#define LED_G 25
#define LED_Y 26
#define LED_R 27

// Wi-Fi (Wokwi simula conexión automáticamente)
const char* ssid = "Wokwi-GUEST";
const char* password = "";

// Broker MQTT público
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
// IMPORTANTE: Cambiar TU-NOMBRE por tu nombre real
const char* topic = "iot-fpuna/TU-NOMBRE/sensor";

WiFiClient espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq = 0;

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(LED_G, OUTPUT);
  pinMode(LED_Y, OUTPUT);
  pinMode(LED_R, OUTPUT);

  // Conectar Wi-Fi
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
  }

  Serial.println("\nWi-Fi conectado");

  // Conectar MQTT
  mqtt.setServer(mqtt_server, mqtt_port);
  while (!mqtt.connected()) {
    if (mqtt.connect("esp32-iot-fpuna")) {
      Serial.println("Conectado al broker MQTT");
    } else { delay(2000); }
  }

  void loop() {
    mqtt.loop();
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    if (isnan(t)) { delay(2000); return; }

    String status = (t>32)?"ALARM":(t>28)?"WARNING":"NORMAL";
    digitalWrite(LED_G, status=="NORMAL");
    digitalWrite(LED_Y, status=="WARNING");
    digitalWrite(LED_R, status=="ALARM");

    seq++;
    // Construir JSON
    String json = "{\"device_id\":\"esp32-01\", \"";
    json += "\"seq\": \"\" + String(seq) + \"\", \"";
    json += "\"temp_c\": \"\" + String(t,1) + \"\", \"";
    json += "\"humidity_pct\": \"\" + String(h,1) + \"\", \"";
    json += "\"status\": \"\" + status + \"\"}";

    // Publicar en MQTT
    mqtt.publish(topic, json.c_str());
    Serial.println("Publicado: " + json);
    delay(3000);
  }
}
```

Explicación Clave

#include <PubSubClient.h>

Librería MQTT para Arduino. Soporta QoS 0 y 1.

WiFi.begin(ssid, password)

En Wokwi, usar "Wokwi-GUEST" y password vacío. En hardware real, poner credenciales de su red.

mqtt.connect("esp32-iot-fpuna")

Client ID: debe ser ÚNICO en el broker. Si dos dispositivos usan el mismo ID, uno se desconecta.

mqtt.publish(topic, json)

Envía el JSON al broker. Todos los suscriptores del topic lo reciben al instante.

mqtt.loop()

OBLIGATORIO en cada iteración del loop. Mantiene viva la conexión MQTT y procesa mensajes entrantes.

broker.hivemq.com

Broker público gratuito para pruebas. En producción, usar broker propio con autenticación y TLS.

2. TECNOLOGÍAS DE RED PARA IoT

Clasificación de Redes IoT – Del Cuerpo a la Nube



BAN

Body Area Network

Alcance: ~2 m

Wearables, sensores médicos corporales. Ejemplo: reloj inteligente, parche de glucosa.



PAN

Personal Area Network

Alcance: ~10 m

Dispositivos personales cercanos. BLE, Zigbee. Ejemplo: auriculares, controlador domótico.



LAN

Local Area Network

Alcance: ~100 m

Red local de un edificio o planta. Wi-Fi, Ethernet. Ejemplo: sensores de fábrica, cámaras.



WAN

Wide Area Network

Alcance: Km+

Redes de largo alcance. LoRaWAN, NB-IoT, 4G/5G, satélite. Ejemplo: agro, ciudades, logística.

Regla de diseño: Elegir la red de menor alcance que cumpla el requisito. Mayor alcance = mayor costo, mayor consumo o menor tasa de datos. En IoT, la eficiencia energética casi siempre domina la decisión.

Redes LPWAN — LoRaWAN y NB-IoT para IoT de Largo Alcance



LoRaWAN (Long Range WAN)

- Tecnología: modulación LoRa (espectro expandido) en banda ISM sin licencia (915 MHz en América).
- Alcance: 2-15 km urbano, hasta 45 km en línea de vista rural.
- Tasa de datos: 0.3-50 Kbps. Suficiente para telemetría pero NO para video o audio.
- Consumo: ultra-bajo. Batería de un sensor puede durar 5-10 años.
- Topología: estrella de estrellas. Gateways LoRa reenvían a un servidor de red central.
- Costo: gateway ~USD 150-300. Módulo sensor ~USD 5-15.
- **Ideal para Paraguay: monitoreo agrícola, medidores de agua en zonas rurales, seguimiento de ganado.**



NB-IoT (Narrowband IoT)

- Tecnología: estándar 3GPP que reutiliza infraestructura celular existente (bandas LTE).
- Alcance: 1-10 km. Excelente penetración en interiores y sótanos.
- Tasa de datos: hasta 250 Kbps (descarga), 20 Kbps (subida).
- Consumo: bajo. No tan eficiente como LoRa pero buena duración de batería (3-5 años).
- Ventaja clave: no requiere infraestructura nueva — las operadoras ya tienen las antenas.
- Costo: módulo ~USD 3-8. Requiere plan IoT-SIM de la operadora.
- **En Paraguay: Claro ya ofrece cobertura NB-IoT/LTE-M.**



¿LoRaWAN o NB-IoT? Guía de Decisión para Paraguay

- LoRaWAN si: necesita cobertura propia (sin depender de operadora), envía pocos datos (<100 bytes), zona sin cobertura celular, quiere control total.
- NB-IoT si: ya tiene cobertura celular, necesita movilidad (tracking), requiere mayor ancho de banda, prefiere no mantener infraestructura propia.
- Híbrido: muchos despliegues usan ambas tecnologías según la zona. LoRaWAN en campo + NB-IoT en área urbana.

Wi-Fi, Bluetooth LE, Zigbee y 5G – Características y Usos en IoT

Wi-Fi (802.11)

Alcance: ~100m | Consumo: Alto | Velocidad: Hasta 9.6 Gbps (Wi-Fi 6)

Ventajas: Alta velocidad, ubicuo, fácil configuración.

Limitaciones: Alto consumo, no ideal para batería, congestión en alta densidad.

Uso IoT: Gateways, cámaras, displays. El ESP32 usa Wi-Fi como canal principal.

Bluetooth LE (BLE 5.0)

Alcance: ~100-400m | Consumo: Muy bajo | Velocidad: 2 Mbps

Ventajas: Ultra-bajo consumo, estándar en celulares, mesh desde BLE 5.0.

Limitaciones: Bajo ancho de banda, alcance limitado en interiores.

Uso IoT: Wearables, beacons, localización indoor, sensores médicos.

Zigbee (802.15.4)

Alcance: ~100m | Consumo: Muy bajo | Velocidad: 250 Kbps

Ventajas: Red mesh autorecuperable, hasta 65,000 nodos, muy bajo consumo.

Limitaciones: Tasa baja, requiere coordinador, ecosistema fragmentado.

Uso IoT: Domótica, automatización de edificios, iluminación inteligente.

5G (NR)

Alcance: ~1-10 km | Consumo: Medio-Alto | Velocidad: Hasta 10+ Gbps

Ventajas: Ultra-baja latencia (<1ms), massive MIMO, network slicing.

Limitaciones: Consumo alto, infraestructura costosa, cobertura limitada aún.

Uso IoT: Robótica remota, vehículos autónomos, AR/VR industrial, IIoT crítico.

3. PLATAFORMAS IOT: NUBE, CÓDIGO ABIERTO Y COMPARATIVA

¿Qué es una Plataforma IoT? – Componentes Esenciales

Una plataforma IoT es el software intermedio (middleware) que conecta los dispositivos con las aplicaciones de negocio. Gestiona la conexión de miles de dispositivos, almacena sus datos, ejecuta reglas y analítica, y expone interfaces para dashboards y aplicaciones.



Los 6 Componentes de una Plataforma IoT



Gestión de Conexiones

Autenticación de dispositivos, gestión de sesiones MQTT/HTTP, balanceo de carga.



Ingesta de Datos

Recepción masiva de mensajes en tiempo real. Colas, buffers y transformación.



Almacenamiento

Bases de datos de series temporales (InfluxDB, TimescaleDB). Almacenamiento caliente y frío.



Procesamiento y Reglas

Motor de reglas para alertas, stream processing, integración con ML y analítica.



Dashboards y APIs

Visualización en tiempo real, APIs REST para integración con ERP/CRM/BI.



Seguridad y Gobierno

Cifrado TLS, gestión de identidades, auditoría, roles y permisos, cumplimiento.

Plataformas IoT Comerciales – AWS, Azure y Google Cloud

AWS IoT Core

Amazon

Plataforma más madura del mercado. Broker MQTT gestionado, reglas, sombras de dispositivos (Device Shadows), integración nativa con Lambda, DynamoDB, S3.

Precio: ~USD 1 por millón de mensajes.

Fortaleza: Ecosistema amplio, escalabilidad masiva, documentación extensa.

Limitación: Curva de aprendizaje alta, costos pueden escalar rápido sin optimización.

Azure IoT Hub

Microsoft

Integración nativa con el ecosistema Microsoft (Power BI, Teams, Active Directory). IoT Edge para despliegue en el borde. Digital Twins integrado.

Precio: Desde ~USD 25/mes (tier básico).

Fortaleza: Excelente para empresas que ya usan Microsoft 365/Azure. Gemelos digitales nativos.

Limitación: Menos flexible que AWS en configuración avanzada. Menor comunidad IoT open source.

Google Cloud IoT

Google

Pub/Sub como broker, BigQuery para analítica masiva, TensorFlow para ML integrado. Google desactivó Cloud IoT Core en 2023 pero mantiene servicios individuales.

Precio: Pay-per-use. BigQuery desde USD 5/TB analizado.

Fortaleza: Mejor opción para analítica y ML. BigQuery es extraordinariamente poderoso.

Limitación: IoT Core discontinuado; requiere ensamblar servicios individuales.

Plataformas IoT de Código Abierto – ThingsBoard, EMQX, Node-RED

ThingsBoard

Plataforma IoT completa: gestión de dispositivos, dashboards, motor de reglas, alertas. Interfaz visual para crear dashboards sin código. Edición Community gratuita y Professional (paga).

En este curso: Ideal para el curso: la usaremos para visualizar datos de los laboratorios.

thingsboard.io

EMQX

Broker MQTT de alto rendimiento. Soporta millones de conexiones simultáneas. Compatible con MQTT 3.1.1 y 5.0. Bridge con Kafka, bases de datos y APIs. Edición open source disponible.

En este curso: Broker MQTT empresarial. Alternativa a Mosquitto para escala industrial.

emqx.io

Mosquitto

Broker MQTT ligero y confiable de Eclipse Foundation. Bajo consumo de recursos, ideal para Raspberry Pi o servidores pequeños. El broker más usado en educación y prototipos.

En este curso: Laboratorios, pruebas de concepto, proyectos con recursos limitados.

mosquitto.org

Node-RED

Herramienta de programación visual basada en flujos (flow-based). Conecta APIs, dispositivos y servicios sin escribir código complejo. Miles de nodos disponibles en su librería.

En este curso: Integración rápida: conectar MQTT con Telegram, email, bases de datos, dashboards.

nodered.org

4. DISEÑO DE ARQUITECTURAS IoT: METODOLOGÍA, PATRONES Y DIAGRAMAS

Metodología para Diseñar una Arquitectura IoT – 6 Pasos

1

Definir el Problema

¿Qué queremos resolver? ¿Quién es el usuario?
¿Cuál es la métrica de éxito? Documentar
requisitos funcionales y no funcionales.

2

Seleccionar Sensores

¿Qué variables físicas medir? Precisión, rango,
frecuencia de muestreo. Costo vs. calidad.
Considerar redundancia.

3

Elegir Conectividad

¿Wi-Fi, BLE, LoRaWAN, NB-IoT, 5G? Depende de:
alcance, consumo, tasa de datos, cobertura,
costo. Aplicar guía de selección.

4

Diseñar Flujo de Datos

Protocolo (MQTT/HTTP), topics, formato del
payload (JSON), frecuencia de envío. Edge vs.
Fog vs. Cloud processing.

5

Seleccionar Plataforma

¿Cloud comercial o código abierto?
¿Almacenamiento, dashboards, reglas, ML?
Evaluar costo, escalabilidad y equipo técnico.

6

Planificar Seguridad

Cifrado TLS, autenticación de dispositivos,
segmentación de red, actualizaciones OTA, plan
de respuesta a incidentes.

Ejemplo: Arquitectura IoT para Monitoreo de Sala de Servidores



Caso de Estudio: El mismo escenario del LAB — Sala de Servidores

CAPA 1: Dispositivos

ESP32 + DHT22 (temp/hum)
Sensor de corriente CT
3 LEDs + Buzzer
Ubicación: Rack A, Sala de servidores

CAPA 2: Red + Protocolo

Wi-Fi 2.4GHz → Router local
Protocolo: MQTT (QoS 1)
Broker: Mosquitto (servidor local)
Topics: empresa/sala-srv/rack-A/temp

CAPA 3: Plataforma

ThingsBoard Community Edition
Almacenamiento: PostgreSQL + TimescaleDB
Motor de reglas: alerta si $T > 32^{\circ}\text{C}$
Dashboard: gráficos en tiempo real

CAPA 4: Aplicación

Dashboard web para el equipo de TI
Alertas por Telegram/Email
Reporte semanal automático (PDF)
Integración futura con CMMS



Decisiones de Diseño y Justificación

¿Por qué ESP32?

Wi-Fi integrado, bajo costo (~USD 4), soporta Arduino y MicroPython, GPIO suficientes.

¿Por qué MQTT y no HTTP?

Bidireccional, cabecera mínima (2 bytes), QoS configurable, LWT para detectar desconexiones.

¿Por qué QoS 1 y no QoS 0?

Las alertas de temperatura son críticas: no podemos perder un mensaje de ALARM. QoS 2 sería excesivo.

¿Por qué ThingsBoard y no AWS?

Costo cero (edición Community), fácil de instalar, dashboards visuales, motor de reglas sin código. AWS sería sobredimensionado para este caso.

¿Y la seguridad?

MQTT con TLS (puerto 8883), usuario/contraseña en el broker, red Wi-Fi aislada (VLAN IoT), firmware firmado.

5. LABORATORIO PBL: MICROPYTHON, THINGSBOARD Y MQTT CON ESP32

Introducción a MicroPython – Python para Microcontroladores

MicroPython es una implementación eficiente del lenguaje Python 3 optimizada para ejecutarse en microcontroladores. Fue creado por Damien George en 2013 mediante una campaña en Kickstarter. Permite programar hardware IoT con la misma sintaxis de Python que se usa en ciencia de datos, web y automatización.



¿Por Qué Usar MicroPython en IoT?

- Prototipado rápido: escribir, probar y modificar sin compilar. El REPL permite ejecutar comandos en tiempo real.
- Curva de aprendizaje baja: si sabés Python, ya sabés MicroPython. Ideal para equipos multidisciplinares.
- JSON nativo: `json.dumps()` y `json.loads()` integrados, sin concatenar strings como en Arduino.
- Librerías IoT: `umqtt.simple` para MQTT, `urequests` para HTTP, `machine` para GPIO/ADC/PWM.
- Compatible con ESP32, ESP8266, Raspberry Pi Pico y STM32.



Arduino IDE vs MicroPython – Resumen

	Arduino IDE	MicroPython
Lenguaje	C/C++	Python 3
Compilación	Sí (flash completo)	No (interpreta al vuelo)
Rendimiento	Muy alto	Moderado
REPL interactivo	No	Sí
Ideal para	Producción	Prototipos, educación



Plataformas que Soportan MicroPython

ESP32

El más usado en IoT. Wi-Fi + BLE integrados. 520KB RAM, dual-core 240MHz.

ESP8266

Predecesor del ESP32. Solo Wi-Fi. Más barato pero menos potente.

Raspberry Pi Pico

ARM Cortex-M0+, 264KB RAM. Sin Wi-Fi nativo (Pico W sí tiene).

STM32

Microcontroladores industriales de alto rendimiento. Menos común en educación.

MicroPython – Sintaxis Básica y Comandos Esenciales para IoT



GPIO, Sensores y Actuadores

```
from machine import Pin, ADC, PWM
import dht, time

# LED como salida digital
led = Pin(25, Pin.OUT)
led.on() # Encender
led.off() # Apagar
led.value(1) # También funciona

# Lectura digital (botón)
boton = Pin(14, Pin.IN, Pin.PULL_UP)
print(boton.value()) # 0 o 1

# Lectura analógica (potenciómetro)
pot = ADC(Pin(34))
pot.atten(ADC.ATTN_11DB) # Rango 0-3.3V
valor = pot.read() # 0-4095

# Sensor DHT22
sensor = dht.DHT22(Pin(4))
sensor.measure()
temp = sensor.temperature() # °C
hum = sensor.humidity() # %
```



Wi-Fi, MQTT y HTTP

```
import network, json
from umqtt.simple import MQTTClient

# Conectar Wi-Fi
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect("Wokwi-GUEST", "")
while not wifi.isconnected():
    time.sleep(0.5)
print("IP:", wifi.ifconfig()[0])

# Conectar MQTT
client = MQTTClient("esp32-01",
    "broker.hivemq.com", port=1883)

client.connect()

# Publicar mensaje
data = {"temp": 28.5, "status": "OK"}
client.publish("iot/sensor",
    json.dumps(data))

# Suscribirse a topic
def callback(topic, msg):
    print("Recibido:", msg)
client.set_callback(callback)
client.subscribe("iot/command")
# En el loop: client.check_msg()
```



Utilidades: JSON, Timers y Deep Sleep

```
import json, time
from machine import deepsleep, Timer

# JSON: serializar y deserializar
datos = {"id": "nodo-01", "temp": 31.2}
texto = json.dumps(datos) # → string
obj = json.loads(texto) # → dict

# Temporización
time.sleep(2) # Esperar 2 seg
time.sleep_ms(500) # Esperar 500 ms
ms = time.ticks_ms() # Milisegundos

# Timer periódico (sin usar delay)
def leer_sensor(timer):
    sensor.measure()
    print(sensor.temperature())
tim = Timer(0)
tim.init(period=3000,
    mode=Timer.PERIODIC,
    callback=leer_sensor)

# Deep sleep (ahorro energético)
deepsleep(60000) # Dormir 60 seg
```



Cheat Sheet: Arduino → MicroPython

Arduino IDE	MicroPython
digitalWrite(pin, HIGH)	pin.on() / pin.value(1)
analogRead(pin)	adc.read()
Serial.println("msg")	print("msg")
delay(2000)	time.sleep(2)
millis()	time.ticks_ms()
String json = "{...}"	json.dumps({...})
#include <PubSubClient.h>	from umqtt.simple import ...

LAB 1 – Montaje del Circuito y Conexión a ThingsBoard



INDIVIDUAL — 30 MIN EN CLASE



Montaje del Circuito en Wokwi — Paso a Paso

1. Abrir wokwi.com → New Project → ESP32.
2. Agregar DHT22: VCC→3V3, GND→GND, DATA→GPIO4.
3. Agregar 3 LEDs con resistencias de 220Ω:
Verde (GPIO25), Amarillo (GPIO26), Rojo (GPIO27).
Ánodo→Resistencia→GPIO | Cátodo→GND.
4. Verificar: todos los GND al bus común del ESP32.
5. En Libraries agregar: DHT sensor library + PubSubClient.
6. El código se conecta a Wi-Fi automáticamente en Wokwi (SSID: "Wokwi-GUEST", password: vacío).



Crear Cuenta Gratuita en ThingsBoard

1. Ir a thingsboard.cloud (versión cloud gratuita).
2. Click en "Sign Up" → registrarse con email.
3. Confirmar email → iniciar sesión.
4. Ir a Entities → Devices → Add Device (+).
5. Nombre: "ESP32-Sala-Servidor". Click en Add.
6. Abrir el dispositivo → pestaña "Credentials".
7. Copiar el Access Token (ej: "A1B2C3D4E5F6").
Este token se usa en el código como autenticación MQTT del dispositivo.



Datos de Conexión MQTT a ThingsBoard Cloud

Servidor: `thingsboard.cloud`
Puerto: `1883` (MQTT sin TLS)
Username: `<tu Access Token>`
Password: `(vacío)`
Topic: `v1/devices/me/telemetry`
Payload: `{"temp":28.5,"humidity":62}`

Diferencia clave vs broker público

En ThingsBoard, el Access Token del dispositivo se envía como username en la conexión MQTT. La contraseña queda vacía. El topic fijo es `v1/devices/me/telemetry`. ThingsBoard recibe el JSON y automáticamente crea las variables del dashboard.

LAB 1 • Código Arduino IDE — ESP32 + MQTT → ThingsBoard

Código Completo — Copiar en sketch.ino de Wokwi

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

#define DHTPIN 4
#define DHTTYPE DHT22
#define LED_G 25
#define LED_Y 26
#define LED_R 27

const char* ssid = "Wokwi-GUEST";
const char* password = "";
// === THINGSBOARD ===
const char* mqtt_server = "thingsboard.cloud";
const int mqtt_port = 1883;
const char* token = "TU_ACCESS_TOKEN"; // Pegar tu token
// Topic fijo de ThingsBoard para telemetría
const char* topic = "v1/devices/me/telemetry";

WiFiClient espClient;
PubSubClient mqtt(espClient);
DHT dht(DHTPIN, DHTTYPE);
int seq = 0;
const float TEMP_WARN = 28.0;
const float TEMP_ALARM = 32.0;

void setup() {
  Serial.begin(115200);
  dht.begin();
  pinMode(LED_G, OUTPUT);
  pinMode(LED_Y, OUTPUT);
  pinMode(LED_R, OUTPUT);

  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
  { delay(500); Serial.print("."); }

  Serial.println("\nWi-Fi OK");

  mqtt.setServer(mqtt_server, mqtt_port);
  while (!mqtt.connected()) {
    // Token como username, password vacío
    if (mqtt.connect("esp32-tb", token, ""))
      Serial.println("ThingsBoard conectado");
    else { Serial.print("Error: ");
      Serial.println(mqtt.state());
      delay(3000); }
  }

  void loop() {
    mqtt.loop();
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    if (isnan(t)) { delay(2000); return; }

    String st = (t >= TEMP_ALARM) ? "ALARM" :
      (t >= TEMP_WARN) ? "WARNING" : "NORMAL";
    digitalWrite(LED_G, st == "NORMAL");
    digitalWrite(LED_Y, st == "WARNING");
    digitalWrite(LED_R, st == "ALARM");
    seq++;

    String json = "{\"temperature\": " + String(t, 1)
      + ", \"humidity\": " + String(h, 1)
      + ", \"status\": \"" + st + "\"}";
    + ", \"seq\": " + String(seq) + "\"";
    mqtt.publish(topic, json.c_str());
    Serial.println("Enviado: " + json);
    delay(3000);
  }
}
```

Notas Clave

`mqtt.connect("esp32-tb", token, "")`

En ThingsBoard el token se pasa como username. El password es vacío (""). El client ID puede ser cualquier string único.

[v1/devices/me/telemetry](#)

Topic FIJO de ThingsBoard. No crear topics propios como en HiveMQ. ThingsBoard lee las claves del JSON y crea las variables automáticamente.

Claves del JSON

Usar "temperature" y "humidity" (sin abreviar) para que ThingsBoard los muestre con nombres claros en el dashboard.

Ver datos en ThingsBoard:

1. Ir a Devices → tu dispositivo
2. Pestaña "Latest telemetry"
3. Verás: temperature, humidity, status, seq actualizándose cada 3 seg

Crear Dashboard:

Dashboards → (+) → Add widget → Charts → Timeseries → seleccionar dispositivo y variable.

LAB 1 • Código MicroPython – ESP32 + MQTT → ThingsBoard

Mismo LAB 1 en MicroPython (alternativa al código Arduino)

```
from machine import Pin
import dht, time, json, network
from umqtt.simple import MQTTClient

# === HARDWARE ===
sensor = dht.DHT22(Pin(4))
led_g = Pin(25, Pin.OUT)
led_y = Pin(26, Pin.OUT)
led_r = Pin(27, Pin.OUT)

TEMP_WARN = 28.0
TEMP_ALARM = 32.0
seq = 0

# === WI-FI ===
wifi = network.WLAN(network.STA_IF)
wifi.active(True)
wifi.connect("Wokwi-GUEST", "")
while not wifi.isconnected():
    time.sleep(0.5)
print("Wi-Fi OK. IP:", wifi.ifconfig()[0])

# === MQTT → THINGSBOARD ===
TOKEN = "TU_ACCESS_TOKEN" # Pegar tu token
client = MQTTClient(
    client_id="esp32-mp",
    server="thingsboard.cloud",
    port=1883,
    user=TOKEN, # Token como usuario
    password=""
)
client.connect()

print("ThingsBoard conectado")

# === LOOP PRINCIPAL ===
while True:
    sensor.measure()
    t = sensor.temperature()
    h = sensor.humidity()
    seq += 1

    if t >= TEMP_ALARM:
        status = "ALARM"
        led_g.off(); led_y.off(); led_r.on()
    elif t >= TEMP_WARN:
        status = "WARNING"
        led_g.off(); led_y.on(); led_r.off()
    else:
        status = "NORMAL"
        led_g.on(); led_y.off(); led_r.off()

    data = {
        "temperature": t,
        "humidity": h,
        "status": status,
        "seq": seq
    }
    client.publish(
        "v1/devices/me/telemetry",
        json.dumps(data)
    )
    print("Enviado:", json.dumps(data))
    time.sleep(3)
```

Diferencias vs Arduino

Estructura

No hay `setup()` ni `loop()`. El código se ejecuta de arriba a abajo. El `while True` reemplaza al `loop()`.

MQTT

Se usa `umqtt.simple` (incluida en MicroPython). El parámetro `user=` recibe el Access Token de ThingsBoard.

JSON

`json.dumps(data)` convierte un diccionario Python a string JSON automáticamente. Mucho más limpio que concatenar en Arduino.

DHT22

En MicroPython, primero se llama `sensor.measure()` y luego se leen los valores con `.temperature()` y `.humidity()` por separado.

En Wokwi:

Para usar MicroPython, crear proyecto ESP32 → seleccionar MicroPython como entorno en la configuración del proyecto. La librería `umqtt.simple` ya está incluida.

LAB 1 • Crear un Dashboard en ThingsBoard — Paso a Paso



Visualizar los Datos del ESP32 en un Dashboard Profesional

1

Verificar telemetría

En ThingsBoard ir a Entities → Devices → tu dispositivo → pestaña "Latest telemetry". Deberías ver temperature, humidity, status y seq actualizándose.

2

Crear Dashboard

Ir a Dashboards → (+) Add Dashboard → Nombre: "Monitor Sala Servidores" → Add. Abrir el dashboard creado → click en el lápiz (editar).

3

Agregar gráfico de temperatura

Click (+) Add widget → Charts → Timeseries - Flot → Add datasource → seleccionar tu dispositivo → key: temperature → Add. Ajustar tamaño arrastrando.

4

Agregar indicador de humedad

(+) Add widget → Analogue gauges → seleccionar tipo gauge → datasource: humidity → rango 0-100 → Add.

5

Agregar tarjeta de estado

(+) Add widget → Cards → Value card → datasource: status → Add. Cambiar color de la tarjeta según el valor (NORMAL=verde, WARNING=amarillo, ALARM=rojo).

6

Guardar y compartir

Click en el ícono de guardar (disquete). El dashboard se actualiza en tiempo real. Podés compartirlo con un enlace público.

LAB 2 – Tarea para EDUCA: Sistema MQTT Bidireccional



TAREA INDIVIDUAL – Entregar Miércoles



ESCENARIO: El gerente de TI quiere control remoto sobre el nodo de monitoreo

Además de visualizar los datos en ThingsBoard, el gerente necesita poder cambiar los umbrales de alerta y encender un ventilador de emergencia desde el dashboard, sin reprogramar el ESP32. Debes implementar comunicación MQTT bidireccional: el ESP32 publica telemetría Y se suscribe a comandos.



Requisitos Técnicos

- 1. Mantener la publicación de telemetría del LAB 1 (temp, humedad, estado) cada 3 segundos.
- 2. Suscribirse al topic de comandos de ThingsBoard: `v1/devices/me/rpc/request/+`
- 3. Al recibir comando `{"method":"setAlarm","params":{"value":35}}`: cambiar el umbral de alarma a 35°C.
- 4. Al recibir `{"method":"fanOn","params":{}}`: encender un LED azul en GPIO33 simulando ventilador.
- 5. Responder al comando en: `v1/devices/me/rpc/response/{requestId}` con `{"result":"OK"}`.
- 6. Agregar potenciómetro (GPIO34) que simule un sensor de corriente eléctrica del ventilador.



Entregables (subir a EDUCA)

- A) Código completo (.ino o .py) funcionando.
- B) Captura de pantalla del Serial Monitor mostrando: conexión exitosa + publicación + recepción de un comando.
- C) Captura del dashboard de ThingsBoard mostrando los datos en tiempo real.
- D) Diagrama de arquitectura del proyecto integrador (Draw.io/Miro) con sensores, protocolo, topics, plataforma y justificación de decisiones (200 palabras).

6. RESUMEN, BIBLIOGRAFÍA Y PRÓXIMOS PASOS

Resumen — Ideas Clave de la Semana 2



4 IDEAS CLAVE QUE DEBES LLEVAR DE HOY

1

MQTT es el protocolo estándar de IoT — y ahora sabemos por qué

Cabecera de 2 bytes, modelo Pub/Sub, QoS configurable, LWT para detectar desconexiones. HTTP es para APIs y dashboards; CoAP es para sensores ultra-restringidos. En el 90% de los casos IoT, MQTT es la respuesta correcta.

2

La elección de red define el alcance, costo y viabilidad del proyecto

LoRaWAN para agro y zonas rurales. NB-IoT donde ya hay cobertura celular. Wi-Fi para interiores. BLE para wearables. 5G para IIoT de ultra-baja latencia. Elegir la red de menor alcance que cumpla el requisito.

3

La plataforma IoT no es solo almacenar datos — es actuar sobre ellos

ThingsBoard, AWS IoT, Azure: todas ofrecen dashboards, reglas y alertas. La decisión depende de escala, presupuesto y equipo técnico. Para prototipos: código abierto. Para producción masiva: cloud comercial.

4

Una buena arquitectura se diseña de abajo hacia arriba con decisiones justificadas

Cada elección (sensor, protocolo, red, plataforma) debe tener una razón técnica y de negocio documentada. El diagrama de arquitectura es el entregable más importante de esta semana.



PRÓXIMOS PASOS: • Semana 3: Gestión de Datos, Analítica y ML en IoT

Referencias

Bibliografía — Semana 2: Conectividad, Plataformas y Arquitecturas

Texto Base

Bahga, A. y Madiseti, V. (2014). Internet of Things: A Hands-On Approach. Capítulos sobre protocolos y red.

Arquitecturas

Buyya, R. y Dastjerdi, A.V. (eds.) (2016). Internet of Things: Principles and Paradigms. Morgan Kaufmann.

Diseño IoT

McEwen, A. y Cassimally, H. (2013). Designing the Internet of Things. John Wiley & Sons.

MQTT Oficial

OASIS Standard. MQTT Version 5.0 (2019). <https://mqtt.org/mqtt-specification/>

LoRaWAN

LoRa Alliance. LoRaWAN Specification v1.0.4 (2020). <https://lora-alliance.org/>

AWS IoT

Amazon Web Services. AWS IoT Core Developer Guide. <https://docs.aws.amazon.com/iot/>

ThingsBoard

ThingsBoard Authors. ThingsBoard Documentation. <https://thingsboard.io/docs/>



Herramientas: wokwi.com · broker.hivemq.com · mqttx.app · thingsboard.io · draw.io · nodered.org · Plataforma EDUCA



¡MUCHAS GRACIAS!